Windows 환경 설치 가이드

이 문서는 Windows에서 Research Vault를 사용하기 위해 필요한 도구들을 설치하는 과정을 안내합니다.

> **참고**: 이 가이드는 **PowerShell** 및 **Git Bash**를 사용하는 네이티브 Windows 환경 기준입니다. Claude Desktop과의 원활한 연동을 위해 WSL 대신 네이티브 터미널 사용을 권장합니다.

목차

1. 사전 준비
2. Git 설치
3. 기본 도구 설치
4. uv 설치
5. fnm + Node.js 설치
6. Claude Desktop 설치
7. Claude Desktop — Connector 설정
8. Claude Desktop — 프로젝트 설정
9. Claude Desktop — Developer MCP 서버 설정
10. Claude Code — MCP 서버 설정
11. Research Vault 설치
12. 설치 확인 체크리스트

---

1. 사전 준비

도구 설치 전에 필요한 외부 서비스 가입과 앱 설치를 먼저 진행합니다.

Obsidian 설치 및 Local REST API 플러그인 활성화

Obsidian은 로컬 마크다운 파일 기반의 노트 앱입니다. Research Vault의 모든 문서를 Obsidian에서 열람하고 관리합니다.

1) Obsidian을 다운로드합니다.

- obsidian.md/download에서 Windows용 설치 파일을 다운로드합니다.

2) 설치 파일을 실행합니다.

다운로드한 `.exe` 파일을 실행하여 설치를 진행합니다.

3) Local REST API 플러그인을 설치합니다.

이 플러그인은 Claude Desktop이 Obsidian vault에 접근할 수 있게 해줍니다.

1. Obsidian 실행
2. 설정 (왼쪽 하단 톱니바퀴) → **커뮤니티 플러그인** → **커뮤니티 플러그인 탐색**
3. "Local REST API" 검색 → **설치** → **활성화**
4. 플러그인 설정에서 **API Key**를 확인하고 복사해 둡니다

> 이 API Key는 나중에 MCP 서버 설정(9단계)에서 사용합니다.

W&B (Weights & Biases) API 키 발급 — 선택

ML 실험 추적을 사용할 경우 W&B API 키를 미리 발급받아 환경 변수에 저장해 둡니다.

1) 계정을 생성합니다.

- wandb.ai에서 가입합니다.

2) API 키를 발급합니다.

- 로그인 후 wandb.ai/authorize 또는 Settings → API Keys에서 키를 복사합니다.

3) 시스템 환경 변수에 저장합니다. (PowerShell에서)

```
[Environment]::SetEnvironmentVariable("WANDB_API_KEY", "여기에_API_키_붙여넣기", "User")
```

4) 터미널을 닫고 새로 열어 저장을 확인합니다.

```
echo $env:WANDB_API_KEY
```

키가 출력되면 성공입니다.

> GUI에서 설정하려면: 시작 메뉴 → "환경 변수" 검색 → **사용자 변수**에 `WANDB_API_KEY` 추가

---

2. Git 설치

Git for Windows는 버전 관리 도구와 함께 **Git Bash** 터미널을 제공합니다. Research Vault의 `setup.sh` 스크립트는 Git Bash에서 실행합니다.

1) PowerShell을 엽니다.

- 시작 메뉴에서 "PowerShell" 검색하여 실행

2) winget으로 Git을 설치합니다.

```
winget install Git.Git
```

> winget이 없는 경우 git-scm.com에서 직접 다운로드합니다.

3) 터미널을 닫고 새로 엽니다. (PATH 갱신을 위해)

4) 설치를 확인합니다.

```
git --version
```

Git Bash 확인

Git 설치 후 시작 메뉴에서 "Git Bash"를 검색하여 실행할 수 있습니다. `setup.sh` 스크립트 실행 시 Git Bash를 사용합니다.

트러블슈팅

증상	해결 방법
`winget`을 찾을 수 없음	Microsoft Store에서 "앱 설치 관리자" 업데이트, 또는 git-scm.com에서 직접 설치
`git: command not found`	터미널을 닫고 새로 열기. 안 되면 시스템 환경 변수에 Git 경로 추가

---

3. 기본 도구 설치

jq는 JSON 데이터 처리 도구입니다. Claude Code hooks에 필요합니다.

```
winget install jqlang.jq
```

설치를 확인합니다:

```
jq --version
```

> `curl`은 Windows 10 이상에 기본 포함되어 있으므로 별도 설치가 불필요합니다.

---

4. uv 설치

uv는 Python 패키지 관리 도구입니다. 기존의 pip, venv, virtualenv를 대체하며, 훨씬 빠른 속도로 패키지를 설치합니다.

1) PowerShell에서 설치 스크립트를 실행합니다.

```
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

2) 터미널을 닫고 새로 엽니다. (PATH 갱신을 위해)

3) 설치를 확인합니다.

```
uv --version
```

버전 번호가 출력되면 성공입니다 (예: `uv 0.6.x`).

트러블슈팅

증상	해결 방법
`uv: command not found`	터미널을 닫고 새로 열기

---

5. fnm + Node.js 설치

fnm(Fast Node Manager)은 Node.js의 여러 버전을 설치하고 전환할 수 있는 도구입니다. Windows PowerShell에서 네이티브로 동작합니다. Claude Code와 일부 MCP 서버 실행에 Node.js가 필요합니다.

1) fnm을 설치합니다.

```
winget install Schniz.fnm
```

2) PowerShell 프로필에 fnm 초기화를 추가합니다.

```
if (!(Test-Path $PROFILE)) { New-Item -Path $PROFILE -Force }
Add-Content $PROFILE 'fnm env --use-on-cd --shell powershell | Out-String
| Invoke-Expression'
```

3) 터미널을 닫고 새로 엽니다.

4) fnm 설치를 확인합니다.

```
fnm --version
```

5) Node.js LTS 버전을 설치합니다.

```
fnm install --lts
fnm use --lts
fnm default lts-latest
```

6) 설치를 확인합니다.

```
node --version
npx --version
```

버전 번호가 출력되면 성공입니다.

7) Node.js 경로를 사용자 PATH에 등록합니다.

Claude Desktop의 MCP 서버에서 `npx`를 사용하려면 PATH에 등록이 필요합니다:

```
$nodePath = (Get-Item (Get-Command node).Source).DirectoryName
[Environment]::SetEnvironmentVariable("PATH",
"$([Environment]::GetEnvironmentVariable('PATH', 'User'));$nodePath",
"User")
```

> Node.js 버전을 변경한 경우 이 단계를 다시 실행해야 합니다.

트러블슈팅

증상	해결 방법
`fnm: command not found`	터미널을 닫고 새로 열기
`node: command not found`	`fnm use --lts` 실행
PowerShell 프로필 실행 오류	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 실행

---

6. Claude Desktop 설치

1) Claude Desktop을 다운로드합니다.

- claude.ai/download에서 Windows용 설치 파일을 다운로드합니다.

2) 설치 파일을 실행합니다.

다운로드한 `.exe` 파일을 실행하여 설치를 진행합니다.

3) 로그인합니다.

Claude 계정으로 로그인합니다. 유료 구독(Pro/Team/Enterprise)이 필요합니다.

---

7. Claude Desktop — Connector 설정

Connector는 Claude Desktop 앱의 UI에서 직접 활성화하는 내장 연동 기능입니다. 설정 파일을 편집할 필요 없이 클릭만으로 연결됩니다.

설정 방법

1. Claude Desktop 앱 열기
2. 설정 (Settings) 진입

3. **Integrations** 또는 **Connectors** 메뉴에서 필요한 서비스를 활성화

권장 Connector

Connector	용도	설정 방법
Filesystem	로컬 파일 읽기/쓰기 (vault 접근)	활성화 후 접근 허용할 디렉토리 경로 추가
Notion	연구 결론 아카이빙	Notion 계정 로그인으로 연결
PubMed	학술 논문 검색	활성화만 하면 사용 가능
bioRxiv	생물학 프리프린트 검색	활성화만 하면 사용 가능

> Filesystem connector에는 vault가 설치될 경로를 추가해야 합니다 (예: `C:\Users\username\Documents\vault`).

---

8. Claude Desktop — 프로젝트 설정

Claude Desktop의 **Project** 기능을 사용하면 대화마다 자동으로 적용되는 지시문(instruction)을 설정할 수 있습니다. Research Vault 전용 프로젝트를 만들어 연구 어시스턴트 규칙을 등록합니다.

프로젝트 생성

1. Claude Desktop 앱 열기
2. 좌측 사이드바에서 **Projects** 클릭
3. **Create Project** 클릭
4. 프로젝트 이름 입력 (예: `Research Vault`)

Instruction 추가

1. 생성한 프로젝트 페이지에서 **Set custom instructions** 클릭
2. Research Vault 저장소의 `docs/claude_desktop_project_instruction_example.md` 내용을 복사하여 붙여넣기
3. **경로를 본인의 vault 경로로 수정합니다:**

수정이 필요한 부분:

```
# 수정 전 (예시)
→ Filesystem:read_text_file path=/Users/yunhakoh/Documents/vault/CLAUDE.md
base path: /Users/yunhakoh/Documents/vault

# 수정 후 (본인 경로로 변경)
→ Filesystem:read_text_file path=C:/Users/본인계정/Documents/vault/CLAUDE.md
base path: C:/Users/본인계정/Documents/vault
```

> 경로는 `setup.sh install`에서 지정한 vault 경로와 동일해야 합니다. Windows에서는 슬래시(`/`) 또는 역슬래시(`\`) 모두 사용 가능합니다.

4. **Save** 클릭

사용 방법

프로젝트 내에서 새 대화를 시작하면 설정한 instruction이 자동으로 적용됩니다. 프로젝트 페이지에서 **Start new chat**을 클릭하거나, 새 대화 시작 시 해당 프로젝트를 선택합니다.

---

9. Claude Desktop — Developer MCP 서버 설정

Developer MCP 서버는 설정 파일(`claude_desktop_config.json`)을 직접 편집하여 연결하는 외부 서버입니다. Connector로 제공되지 않는 기능을 추가할 때 사용합니다.

설정 파일 위치

```
%APPDATA%\Claude\claude_desktop_config.json
```

설정 파일이 없으면 새로 생성합니다.

설정 파일 기본 구조

```
{
    "mcpServers": {

    }
}
```

`mcpServers` 안에 각 서버를 추가합니다. 아래의 필요한 서버를 골라서 추가하세요.

mcp-obsidian (Vault 연동 — 권장)

Claude Desktop에서 Obsidian vault의 내용을 검색하고 수정할 수 있게 합니다.

사전 요구사항: Obsidian의 Local REST API 플러그인이 설치·활성화되어 있어야 합니다 (1단계 참조).

```
{
    "mcpServers": {
        "mcp-obsidian": {
            "command": "uvx",
            "args": ["mcp-obsidian"],
            "env": {
                "OBSIDIAN_API_KEY": "여기에_Local_REST_API_키를_붙여넣기"
            }
        }
    }
}
```

> `OBSIDIAN_API_KEY`는 Obsidian → 설정 → Local REST API 플러그인 → API Key에서 확인할 수 있습니다.

`uvx`를 못 찾는 경우, 전체 경로를 사용합니다:

```
"command": "C:\\Users\\username\\.local\\bin\\uvx.exe"
```

경로 확인: PowerShell에서 `where.exe uvx`

W&B (실험 추적 — 선택)

W&B(Weights & Biases)로 ML 실험 결과를 조회할 수 있게 합니다.

사전 요구사항: W&B API 키가 시스템 환경 변수에 설정되어 있어야 합니다 (1단계 참조).

```
{
    "mcpServers": {
        "wandb": {
            "command": "uvx",
            "args": [
                "--from", "git+https://github.com/wandb/wandb-mcp-server",
                "wandb_mcp_server"
            ],
            "env": {
                "WANDB_API_KEY": "여기에_W&B_API_키를_붙여넣기"
            }
        }
    }
}
```

> 시스템 환경 변수에 `WANDB_API_KEY`를 설정한 경우에도, Claude Desktop은 시스템 환경 변수를 상속하지 않을 수 있으므로 위 설정에 키를 직접 입력하는 것을 권장합니다.

Context7 (라이브러리 문서 조회 — 선택)

최신 라이브러리 문서를 Claude에서 바로 검색할 수 있게 합니다.

```
{
    "mcpServers": {
        "context7": {
            "command": "npx",
            "args": ["-y", "@upstash/context7-mcp"]
        }
    }
}
```

여러 서버를 함께 설정하는 예시

```
{
    "mcpServers": {
        "mcp-obsidian": {
            "command": "uvx",
            "args": ["mcp-obsidian"],
            "env": {
                "OBSIDIAN_API_KEY": "your-api-key"
            }
        },
        "context7": {
            "command": "npx",
            "args": ["-y", "@upstash/context7-mcp"]
        }
    }
}
```

> 설정 파일 수정 후 **Claude Desktop을 재시작**해야 적용됩니다.

트러블슈팅

증상	해결 방법
MCP 서버가 연결되지 않음	Claude Desktop을 완전히 종료 후 재시작
uvx를 찾을 수 없음	command에 전체 경로 사용 — PowerShell에서 `where.exe uvx`로 확인
npx를 찾을 수 없음	5단계의 7번(PATH 등록)을 확인하거나, command에 전체 경로 사용 — PowerShell에서 `where.exe npx`로 확인
JSON 문법 오류	쉼표, 중괄호, 따옴표가 올바른지 확인
Obsidian 연결 실패	Obsidian이 실행 중이고, Local REST API 플러그인이 활성화되어 있는지 확인

---

10. Claude Code — MCP 서버 설정

Claude Code에서 사용할 MCP 서버는 `claude mcp add` 명령어로 등록합니다. Git Bash에서 실행합니다.

> 아래 서버들은 Research Vault의 `setup.sh` 스크립트로도 자동 설정할 수 있습니다 (11단계 참조).

Notion (연구 아카이빙)

```
claude mcp add notion --scope user -- npx -y @anthropic/notion-mcp-server
```

bioRxiv (프리프린트 검색)

```
claude mcp add bioRxiv --scope user -- npx -y @anthropic/biorxiv-mcp-
server
```

PubMed (논문 검색)

```
claude mcp add PubMed --scope user -- npx -y @anthropic/pubmed-mcp-server
```

등록 확인

```
claude mcp list
```

등록된 MCP 서버 목록이 출력됩니다.

> **참고**: mcp-obsidian, W&B, Context7은 `setup.sh obsidian`, `setup.sh wandb`, `setup.sh context7` 명령으로 자동 등록됩니다. 아래 Research Vault 설치 과정에서 안내합니다.

---

11. Research Vault 설치

> 이 섹션의 모든 명령은 **Git Bash**에서 실행합니다. 시작 메뉴에서 "Git Bash"를 검색하여 실행하세요.

저장소 클론

```
git clone git@github.com:yunhak0/research-vault.git
cd research-vault
```

> SSH key가 설정되지 않은 경우 HTTPS를 사용합니다:
>
> ```
> git clone https://github.com/yunhak0/research-vault.git
> ```

Vault 초기 설치

```
bash setup.sh install
```

대화형으로 다음을 설정합니다:

- Vault 설치 경로 (예: `C:\Users\username\Documents\vault`)
- 소유자 이름
- Notion 연동 여부

첫 프로젝트 생성

```
bash setup.sh project
```

프로젝트 ID, 이름, 코드 저장소 경로를 입력합니다.

Daily 자동화 설치

```
bash setup.sh daily
```

셸에 `daily-start`, `daily-update` alias를 등록합니다. 설치 후 셸을 다시 불러옵니다:

```
source ~/.bashrc
```

Obsidian에서 Vault 열기

1. Obsidian 실행
2. **Open folder as vault** 선택
3. `setup.sh install`에서 지정한 vault 경로를 선택

---

12. 설치 확인 체크리스트

모든 설치가 완료되었는지 아래 항목을 확인합니다.

기본 도구

- [ ] `uv --version` — 버전 출력 확인
- [ ] `node --version` — 버전 출력 확인
- [ ] `npx --version` — 버전 출력 확인

Claude Desktop

- [ ] 앱 실행 및 로그인 성공
- [ ] Connector (Filesystem, Notion 등) 활성화 확인
- [ ] Developer MCP 서버 연결 확인 (설정한 경우)

Obsidian

- [ ] Obsidian에서 vault 열기 성공
- [ ] Local REST API 플러그인 활성화 확인

Research Vault

- [ ] `daily-start` 명령 동작 확인
- [ ] Claude Desktop에서 vault 문서 접근 가능 확인

모든 항목이 체크되었다면 Research Vault 사용 준비가 완료된 것입니다.